

Learning Objectives

In the videos, we developed several basic building blocks of digital circuits, including binary arithmetic, two's-complement representation, adders, multiplexers, and combinational logic.

In **Worksheets 4 and 6**, we will combine these ideas into a small processor datapath that can eventually be controlled using **8-bit machine instructions**. It is not yet a complete processor: the instructions are entered manually. A complete processor also needs a way to store instructions and to fetch and execute them automatically. We will get there later.

We will not yet know how to build every component we need. That is intentional. Before looking at all the details, it is useful to know where we are going. In Worksheet 2, you already got a first glimpse of sequential logic and experimented with D latches and D flip-flops. You therefore also have a rough idea of how several D flip-flops can be combined to store a multi-bit pattern — for example, to build an 8-bit register.

In this worksheet, however, we deliberately return to combinational logic and concentrate on the parts of the Mini-ALU that do not need to remember any state.

The lecture videos before Worksheet 6 will return to sequential logic in more detail and answer the questions that we deliberately left open in Worksheet 2. In Worksheet 6, we will then put these ideas into practice by building the 8-bit registers and combining them with the components developed here.

By the end of the two worksheets, the resulting Mini-ALU will contain

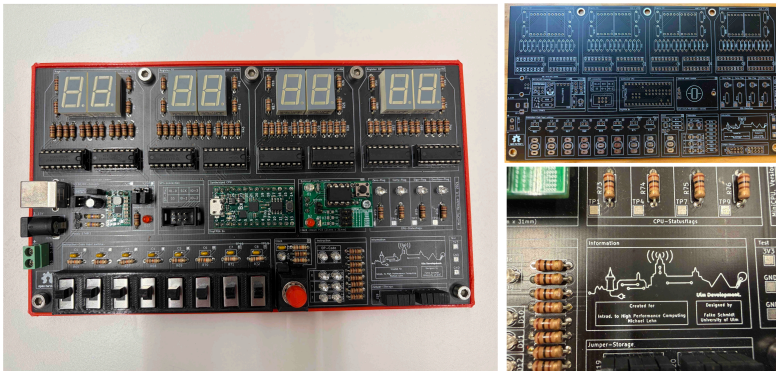
- an 8-bit arithmetic unit,
- status registers,
- four 8-bit registers, and
- an instruction decoder.

Instead of controlling all these components individually, we will then control the circuit using 8-bit machine code.

The Real Thing

Everything you build in CircuitVerse also exists as physical hardware in the classroom.

The board shown below implements the processor datapath that we will develop in Worksheets 4 and 6. It has four 8-bit registers, whose contents are displayed in hexadecimal on the four displays at the top.



At the bottom are eight switches. Together, they represent one 8-bit machine instruction. Set the switches to the desired bit pattern and press the red button: the instruction is executed.

For example, to compute $3 + 2$, you can enter the following three instructions:

```
00001101    load 3 into register %1
00001001    load 2 into register %2
10011011    add %1 and %2 and store the result in %3
```

After each instruction, press the red button once. You can watch the contents of the registers change on the displays.

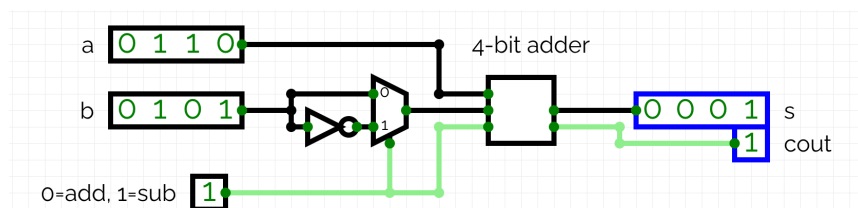
You can then continue with

```
11011011    subtract %1 from %2 and store the result in %3
```

What should now be displayed for register %3? And which status flags should be set? You should be able to answer both questions by the end of this project. ;-)

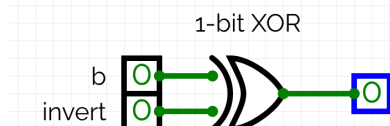
Step 1 — Simplifying the Adder/Subtractor

In the videos, we constructed a 4-bit adder/subtractor using two's-complement arithmetic. To switch between addition and subtraction, we used an inverter and a multiplexer: for addition, the bits of **b** were passed through unchanged; for subtraction, their inverted values were passed to the adder.



This works—but we can do the same thing with *less hardware*. In Worksheet 2, you already encountered a useful XOR trick: one input of an XOR gate can be used to control whether the other input is passed through unchanged or inverted. We used this idea to switch between addition and subtraction. You have therefore seen the basic idea before. In this step, we revisit it and use it systematically to simplify our 8-bit adder/subtractor.

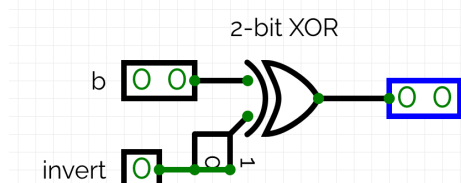
Consider first a single XOR gate with the inputs **b** and **invert**.



Task: Rebuild the circuit shown below in CircuitVerse and complete its truth table.

b	invert	output
0	0	
1	0	
0	1	
1	1	

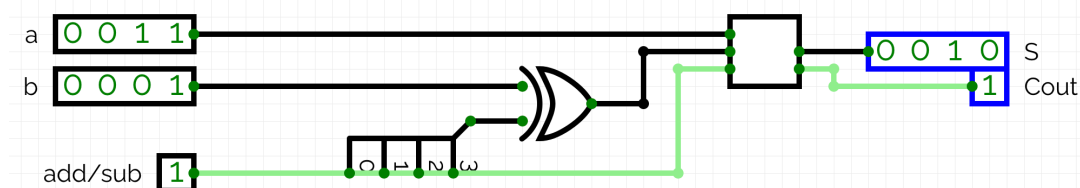
Now extend the idea to a 2-bit value **b**. Change the data width of the XOR gate to 2 bits and use a splitter to connect the single **invert** input to both bits, as shown below.



Experiment with different values of **b** for **invert** = 0 and **invert** = 1.

Task: Now take another look at the 4-bit adder/subtractor shown above. Use what you have just observed to simplify this circuit. Your new circuit should have the same inputs and outputs and behave exactly like the original one, but require fewer components.

Once your simplified 4-bit version works, extend it to 8 bits.



Step 2 – Status Flags

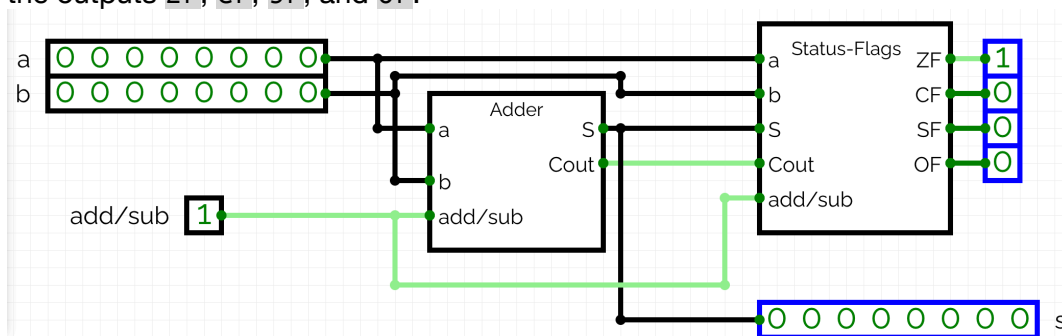
The videos showed how status flags can be used to obtain additional information about the result of an addition or subtraction. In particular, you saw how the relevant flags can be determined from the operands and the result.

This step is therefore mainly a review: you will now turn these ideas into a CircuitVerse circuit. We use the following four flags:

- **ZF (Zero Flag):** 1 if the result s consists entirely of zero bits; otherwise 0.
- **CF (Carry Flag):** indicates an unsigned overflow for addition or an unsigned borrow for subtraction.
- **OF (Overflow Flag):** 1 if the result cannot be represented as an 8-bit signed integer.
- **SF (Sign Flag):** 1 if the result, interpreted as an 8-bit signed integer, is negative.

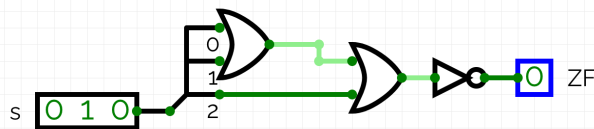
Task

Create a CircuitVerse module called Status Flags with the inputs a , b , s , $Cout$, and add/sub , and the outputs ZF , CF , SF , and OF .



Hint for ZF

The circuit below shows how the Zero Flag can be computed for a 3-bit result s .



A splitter separates the three bits of s . The OR gates determine whether at least one of these bits is 1; the final inverter therefore produces 1 exactly when all three bits are 0.

Use the same idea to implement ZF for the 8-bit result of your adder/subtractor. You will need more OR gates, but no new idea.

Hint for CF

In the videos, it was explained why $Cout$ can be used directly as CF for addition, while it has to be inverted for subtraction.

Before implementing CF , explain the reason once more — either to me or to your neighbor. Being able to build the circuit is not enough; you should also be able to explain why it works.

And yes, this is exactly the kind of question that might come up in the oral exam. ;-)

Once this is clear, the XOR trick revisited in Step 1 gives you a particularly simple implementation.

Hint for SF

Use a splitter to extract the most significant bit of s and connect it directly to SF .

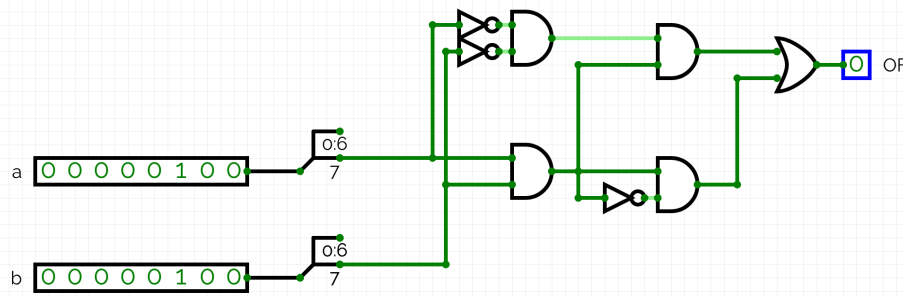
Hint for OF

Signed overflow for addition was discussed in the videos. Recall the result:

If a , b , and S are interpreted as signed integers, OF is set exactly in either of the following two cases:

1. a and b are negative, but s is non-negative.
2. a and b are non-negative, but s is negative.

Therefore, only the most significant bits of a , b , and S are needed. Use splitters to extract these bits and implement the two conditions above using logic gates.



For subtraction, remember that $a \ominus b = a \oplus \bar{b} + 1$. Compared with addition, the role of the sign bit of b is therefore reversed. Extend your circuit so that it uses the most significant bit of b for addition and its inverted value for subtraction.

The same XOR trick is useful once again here.

Step 3 – Zero Extension and Left Shift

In the videos, we introduced the notation

$$u(a_{n-1}, \dots, a_0) = \sum_{\ell=0}^{n-1} a_{\ell} 2^{\ell}$$

for the unsigned integer represented by the bit pattern. We follow Donald Knuth's notation here. So complaints about the notation should be addressed accordingly. ;-)

In this step, we will build two very simple CircuitVerse modules. Instead of describing first how the circuits should be constructed, we specify their behavior mathematically. Your task is to translate these specifications into hardware.

Zero Extension

Build a module `ZeroExtend` with a 4-bit input `in` and an 8-bit output `out`



such that

$$u(in_3, in_2, in_1, in_0) = u(out_7, out_6, out_5, out_4, out_3, out_2, out_1, out_0)$$

In other words, the 8-bit output must represent exactly the same unsigned integer as the 4-bit input.

Task: Implement `ZeroExtend` using splitters and constant inputs. Test your module with several different input patterns.

Left Shift

Build a module `LeftShift` with a 4-bit input `in` and an 8-bit output `out`



such that

$$u(in_3, in_2, in_1, in_0) \cdot 2^4 = u(out_7, out_6, out_5, out_4, out_3, out_2, out_1, out_0)$$

Task: Implement `LeftShift` using splitters and constant inputs and test it with several different input patterns.

Note: Both modules are extremely simple. The point of this exercise is not the complexity of the circuits, but the translation of a mathematical specification into a concrete hardware implementation.

Step 4 — An 8-bit Instruction Format

The physical Mini-ALU can be controlled using the eight switches at the bottom of the board. Together, these switches specify a single 8-bit machine instruction. Pressing the red button executes this instruction.

We will now introduce the instruction format used by the Mini-ALU.

For an 8-bit instruction

$(instr_7, instr_6, instr_5, instr_4, instr_3, instr_2, instr_1, instr_0)$

we introduce the following abbreviations:

$Op = (instr_7, instr_6),$

$X = (instr_5, instr_4),$

$Y = (instr_3, instr_2),$

$Z = (instr_1, instr_0),$

and additionally

$XY = (instr_5, instr_4, instr_3, instr_2).$

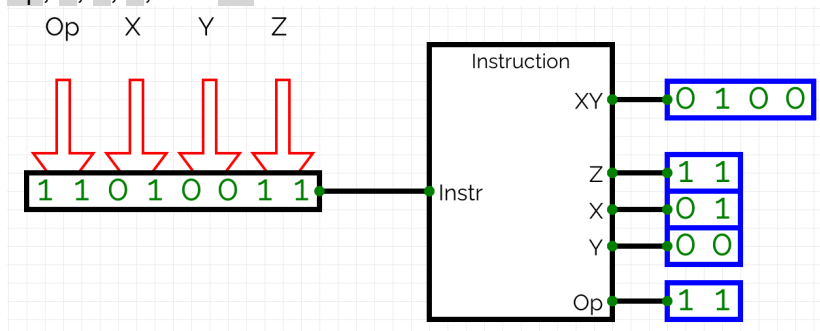
Op is the *operation code*, or *opcode* for short. It determines which operation is performed. The meaning of **X**, **Y**, **Z**, and **XY** depends on the opcode.

The notation is shamelessly borrowed from Donald Knuth. ;-)

Extracting the Bit Fields

Despite all the notation, there is nothing complicated to build yet. The module **Instruction** simply uses splitters to extract the corresponding bits from the 8-bit instruction.

Task: Build the **Instruction** module shown below. It has one 8-bit input **Instr** and the outputs **Op**, **X**, **Y**, **Z**, and **XY**.



Test your module, for example, with

Instr = 11010011

You should obtain

Op = 11, **X** = 01, **Y** = 00, **Z** = 11, **XY** = 0100.

A First Look Ahead

In the videos before Worksheet 6, you will learn about flip-flops. We will use them to build registers that can store 8-bit patterns. Our Mini-ALU will have four such registers, called

%0, **%1**, **%2**, **%3**.

Since **X**, **Y**, and **Z** are two bits wide, each of them can identify one of these four registers.

You do not need to know yet how a register works.

For now, simply think of it as a small box that can remember eight bits.

With this in mind, we can already specify what our 8-bit instructions will do.

`Op = 00`

For

`Op = 00`

the instruction performs

$u(XY) \rightarrow u(\%Z)$

Here `%Z` denotes the register selected by `Z`. We call this the destination register.

Since `XY` contains only four bits while a register contains eight bits, `XY` is extended with zeros on the left before it is stored. This is exactly what the ZeroExtend module from Step 3 does.

For example, consider

`Instr = 00001101`

Splitting the instruction into its fields gives

<u>00</u>	<u>00</u>	<u>11</u>	<u>01</u>
=Op	=X	=Y	=Z

Thus,

`XY = 0011`, `Z = 01`.

The instruction therefore stores the bit pattern

`00000011`

in register `%1`. After executing the instruction,

$u(\%1) = 3$.

You can already try this instruction on the physical Mini-ALU: set the eight switches to `00001101` and press the red button.

`Op = 01`

For

`Op = 01`

the instruction performs

$(u(XY) \cdot 2^4 + u(\%Z)) \bmod 2^8 \rightarrow u(\%Z)$

This looks more complicated than it actually is.

The expression

$u(XY) \cdot 2^4$

means that the 4-bit pattern `XY` is shifted four positions to the left. This is exactly what the module `LeftShift` from Step 3 does.

The shifted value is then added to the current value of the destination register `%Z`. The modulo 2^8 means that only the lower eight bits of the result are kept.

Together, instructions with `Op = 00` and `Op = 01` allow us to construct an arbitrary 8-bit value in a register:

- `Op = 00` sets the lower four bits.
- `Op = 01` sets the upper four bits.

The somewhat unusual use of addition for `Op = 01` is intentional: it will make the hardware particularly simple.

Op = 10

For

Op = 10

the instruction performs

$$(u(\%X) + u(\%Y)) \bmod 2^8 \rightarrow u(\%Z)$$

Thus, the contents of registers %X and %Y are added, and the 8-bit result is stored in the destination register %Z.

For example,

10011011

gives

Op = 10, X = 01, Y = 10, Z = 11.

It therefore adds the contents of %1 and %2 and stores the result in %3.

Op = 11

For

Op = 11

the instruction performs

$$(-u(\%X) + u(\%Y)) \bmod 2^8 \rightarrow u(\%Z)$$

Thus, the content of register %X gets **subtracted from** the content of register %Y, and the 8-bit result is stored in the destination register %Z.

For example,

11011011

gives

Op = 11, X = 01, Y = 10, Z = 11,

and therefore computes

$$(-u(\%1) + u(\%2)) \bmod 2^8 \rightarrow u(\%3)$$

In other words: subtract %1 from %2 and store the result in %3.

Summary

Op	Operation
00	$u(XY) \rightarrow u(\%Z)$
01	$(u(XY) \cdot 2^4 + u(\%Z)) \bmod 2^8 \rightarrow u(\%Z)$
10	$(u(\%X) + u(\%Y)) \bmod 2^8 \rightarrow u(\%Z)$
11	$(-u(\%X) + u(\%Y)) \bmod 2^8 \rightarrow u(\%Z)$

You can already enter these instructions manually on the physical Mini-ALU and observe their effect. After each addition or subtraction, the four status flags ZF, CF, SF, and OF that you implemented in Step 2 are also updated and stored in status registers.

In Worksheet 6, after introducing flip-flops and registers, we will build the circuitry that makes all of this work.

A First Look at SystemVerilog

So far, we have described digital circuits graphically in CircuitVerse. As circuits become larger, however, placing gates and connecting wires manually becomes increasingly cumbersome.

There is another way to describe exactly the same hardware: text.

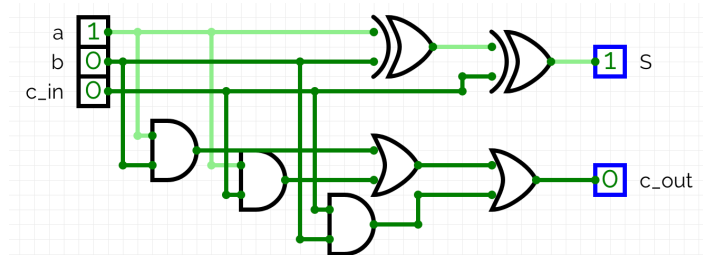
A *Hardware Description Language (HDL)* provides another way to describe the same hardware — as text. Here we take a first look at *SystemVerilog*.

You are not expected to learn SystemVerilog in this worksheet. The goal is simply to see how closely a textual hardware description corresponds to the logic circuits you already know.

Example: A Full Adder

In the videos, you constructed a full adder from two half adders. Convince yourself that the same functionality can also be described directly by

$$s = a \dot{\vee} b \dot{\vee} c_{in}, \quad c_{out} = (a \wedge b) \vee (a \wedge c_{in}) \vee (b \wedge c_{in}).$$



The following SystemVerilog module describes this circuit and connects it to three buttons and two LEDs of an FPGA board:

```
module test (
    input  logic BTN1,
    input  logic BTN2,
    input  logic BTN3,
    output logic LED1,
    output logic LED2
);

    logic a, b, c_in;
    assign a = BTN1;
    assign b = BTN2;
    assign c_in = BTN3;

    logic s, c_out;
    assign LED1 = s;
    assign LED2 = c_out;

    always_comb begin
        s      = a ^ b ^ c_in;
        c_out = (a & b) | (a & c_in) | (b & c_in);
    end
endmodule
```

Boolean Operators

The operators used here correspond directly to Boolean operations you already know:

SystemVerilog	Boolean logic	Meaning
&	$\wedge$	AND
	$\vee$	OR
^	$\dot{\vee}$	XOR
~	$\neg$	NOT

Connecting to the Outside World

The names `BTN1`, `BTN2`, `BTN3`, `LED1`, and `LED2` refer to signals connected to physical components on our FPGA board.

For the description of the actual logic, however, the names `a`, `b`, `c_in`, `s`, and `c_out` are more convenient. The `assign` statements connect/wire these internal names to the outside world:

```
assign a = BTN1;
```

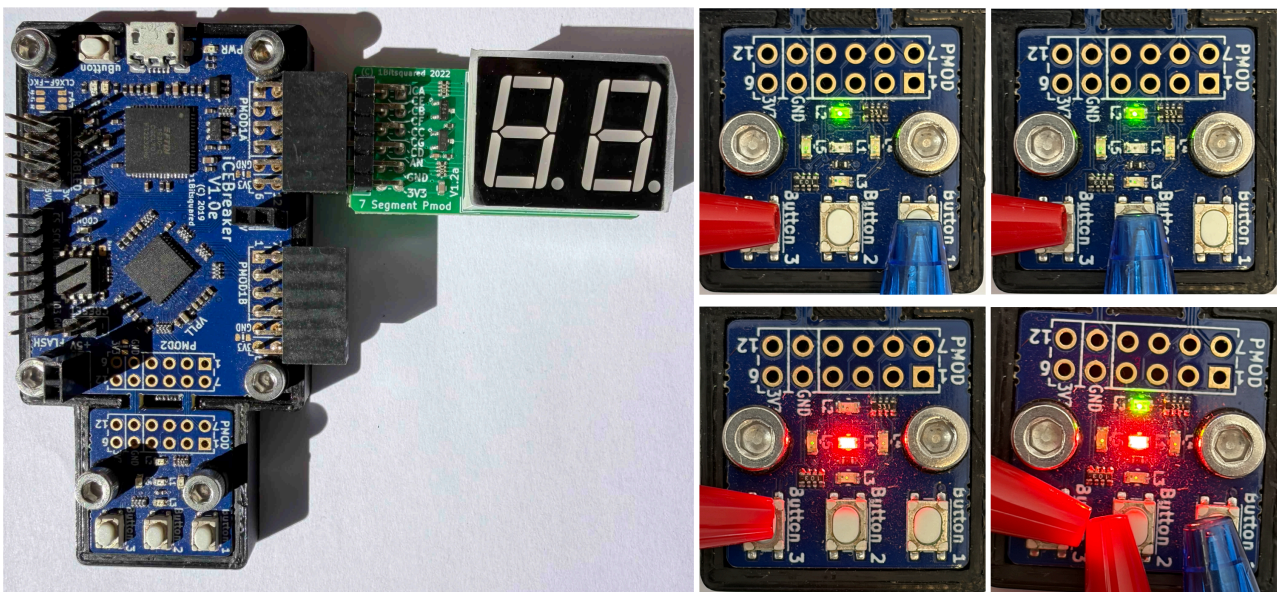
```
assign LED1 = s;
```

You can think of these statements as giving the external signals convenient internal names.

This Describes Hardware

Important: SystemVerilog may look like a programming language, but the code above is not a program that executes the statements one after another. It is a *textual description of the logic circuit* shown above.

The `always_comb` block describes combinational hardware: the outputs continuously depend on the inputs according to the Boolean expressions.



Full Adder — iCE40 FPGA

The SystemVerilog description can be synthesized for an **iCE40 FPGA** and loaded onto the board shown above. The buttons then provide `a`, `b`, and `c_in`, while the LEDs display `s` and `c_out`.

The physical Mini-ALU used in this course is built in the same way: its hardware is described in SystemVerilog and implemented on an FPGA.